01_Cleaning

Contents


```
# Load packages
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ggplot2)
library(readxl)
library(lubridate)
library(janitor)
```

```
##
## Attaching package: 'janitor'
##
## The following objects are masked from 'package:stats':
##
##     chisq.test, fisher.test
```

```
# Global options
knitr::opts_chunk$set(echo = TRUE, warning = FALSE, message = FALSE)
```

Introduction

This script will contain the code necessary to clean the data stored in the `/RawData` folder.

To start it will focus on only cleaning data puled pertaining to the 9000 IMIS classification; later on, subsequent datasets may be cleaned, merged, and analyzed.

New variables created during the cleaning process, as well as a description of all other included variables, are contained within the excel file `DataDictionary.xlsx`.

Goals of Cleaning:

1. Ensure that UOM units are consistent
2. Apply inclusion/exclusion criteria
3. Add OEL data, create variable that states whether observed sample is above or below.

Data Import and Initial Examination

```
dat <- read_delim("../RawData/Respirable_Silica_Data.txt", delim = "|")
glimpse(dat)
head(dat)
tail(dat)
```

There are a couple of formatting tasks we need to take care of. We want to convert our data into the proper formats, and there are several variables included in the dataset that we will not need (Office, NAICS, Date_Reported, Lab_Number, CSHO).

We want to achieve the following: - Convert 'result' to be numeric. *Note:* some of these values are recorded as > measurement. To deal with this, we will make a new variable, `result_num` that will report whatever number is associated with the measurement. Another variable, `less_than_measurement`, will indicate whether the variable was "less than" or not.
- Convert date to an R date format - Create a new variable that just contains the year of sample - Converting `time` to be in terms of hours, not minutes

Convert result to numeric, check how many are blanks? Also check how many are per sample type and make into nicer format. Check how many went in and used eight hour TWA calculation too. Changing time to be in terms of hours instead of minutes. Adding indicator in case of data being "less than" values, so we can go back in and perhaps do a sensitivity analysis of some kind or just have the knowledge that they were less than that number.

To start, let's simply apply some reformats to our data:

```
dat <- dat |>
  rename_with(tolower) |>
  mutate(
    date_sampled = dmy(date_sampled),
    year = year(date_sampled),
         type = fct_recode(type,
                           "Personal" = "P",
                           "Area" = "A",
                           "Bulk" = "B",
                           "Wipe" = "W"),
```

```
    blank = as.logical(blank),
    time = time/60,
    less_than_measurement = grepl("^\\s*<", result),
    result_num = as.numeric(gsub("<\\s*", "", result))
    ) |>
  select(-office, -naics, -date_reported, -lab_number, -csho, -sic)
```

Examination

We want to check some of these variables to see whether they are relevant to be included (i.e., contain multiple levels).

Let's check what different values are in the data for `instrument`, `blank`, `weight`, and `uom`.

```
dat |>
  group_by(instrument) |>
  summarize(n())
```

```
## # A tibble: 1 x 2
##   instrument `n()`
##   <lgl>      <int>
## 1 NA          2135
```

```
dat |>
  group_by(blank) |>
  summarize(n())
```

```
## # A tibble: 1 x 2
##   blank `n()`
##   <lgl> <int>
## 1 NA     2135
```

```
summary(dat$weight)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NAs
##  0.0000  0.0150  0.0530  0.2887  0.1675 16.1420     569
```

```
dat |>
  group_by(uom) |>
  summarize(n())
```

```
## # A tibble: 7 x 2
##   uom    `n()`
##   <chr>  <int>
## 1 M        202
## 2 MCG/M3  1236
## 3 MG       123
## 4 MG_M3     69
## 5 X        306
## 6 Y        143
## 7 <NA>      56
```

It looks like we only have NA values for instrument and blank; we can drop. We do have information for weight that may be needed.

For the unit of measurement, according to the data dictionary, M = mg/mˆ3, X = micrograms, Y = milligrams. We have 56 NAs which we will need to explore – it is possible it corresponds to values that contain sample weight and a result, from which we will need to complete some calculations.

Let's check to see what the NA values correspond to before we reformat the uom variable.

```
dat |>
  filter(is.na(uom)) |>
  select(weight, result_num, qualifier, air_volume)
```

```
## # A tibble: 56 x 4
##    weight result_num qualifier air_volume
##     <dbl>      <dbl> <chr>          <dbl>
##  1  0.124         NA <NA>            680.
##  2  0             NA <NA>             NA
##  3  0.219         NA <NA>            719.
##  4  0.068         NA <NA>            735.
##  5  0.062         NA <NA>            720.
##  6  0             NA <NA>            710.
##  7  0.096         NA <NA>            708.
##  8  0.057         NA <NA>            228.
##  9  0.199         NA <NA>            515.
## 10  0             NA <NA>             NA
## # i 46 more rows
```

This is interesting, since we have measurements for weight, but we don't know what unit the sample is in for the bulks and silica samples. We are likely unable to use these observations.

Another important aspect is that we have some units of measurement that are simply micrograms, and milligrams, without being accompanied with the air volume. Let's check and make sure that all of these instances correspond to an air volume value. If they are all complete, we will need to look into converting the air volume (liters) to meters cubed.

```
dat |>
  filter(uom == c("M", "MG", "X", "Y")) |>
  select(uom, air_volume) |>
  group_by(uom) |>
  summarise(
    total_rows = n(),
    non_na_values = sum(!is.na(air_volume)),
    na_count = sum(is.na(air_volume))
  )
```

```
## # A tibble: 4 x 4
##   uom   total_rows non_na_values na_count
##   <chr>      <int>         <int>    <int>
## 1 M             60            57        3
## 2 MG            30             0       30
## 3 X             96             0       96
## 4 Y             36             0       36
```

It appears that we have some data that is missing air volume specifically. We will need to double check which rows are missing air volume after we exclude non-personal samples, and then look into creating a standardized measurement.

Let's apply some reformatting again with the new information.
M = mg/m^3, X = micrograms, Y = milligrams.

```
dat <- dat |>
  select(-instrument, -blank) |>
  mutate(
    uom = fct_recode(uom,
                           "mg" = "MG", #milligrams
                           "mg" = "Y", #milligrams
                           "mcg" = "X", #micrograms
                           "mcg/m3" = "MCG/M3", #micrograms/m3
                           "mg/m3" = "MG_M3", #milligrams/m3
                           "mg/m3" = "M" #milligrams/m3
                       )
    )


dat |>
  group_by(uom) |>
  summarize(n())
```

```
## # A tibble: 5 x 2
##   uom    `n()`
##   <fct>  <int>
## 1 mg/m3    271
## 2 mcg/m3  1236
## 3 mg       266
## 4 mcg      306
## 5 <NA>      56
```

Inclusion/Exclusion

Personal Measurements

To start, we have 2,135 observations. Next, we want to restrict our sample to only those pertaining to personal measurements.

```
dat |>
  group_by(type) |>
  summarize(n())
```

```
## # A tibble: 3 x 2
##   type     `n()`
##   <fct>    <int>
## 1 Area        20
## 2 Personal  1839
## 3 <NA>       276
```

We will retain 1,839 observations that used Personal measurements. We will drop those for Area measurements (n = 20), and missing data (n = 276).

```
dat <- dat |>
  filter(type == "Personal")
```

Now that we've excluded some samples, let's go back in and see how the air volume data fits or doesn't:

```
dat |>
  filter(uom %in% c("mg", "mcg")) |>
  select(uom, air_volume) |>
  group_by(uom) |>
  summarise(
    total_rows = n(),
    non_na_values = sum(!is.na(air_volume)),
    na_count = sum(is.na(air_volume))
  )
```

```
## # A tibble: 1 x 4
##   uom   total_rows non_na_values na_count
##   <fct>      <int>         <int>    <int>
## 1 mcg          299             0      299
```

Let's also check to see the distribution of samples by year:

```
dat |>
  group_by(year) |>
  summarize(count = n()) |>
  mutate(percent = (count/sum(count))*100)
```

```
## # A tibble: 10 x 3
##     year count percent
##    <dbl> <int>   <dbl>
##  1  2017    46    2.50
##  2  2018    67    3.64
##  3  2019   154    8.37
##  4  2020    89    4.84
##  5  2021   144    7.83
##  6  2022   107    5.82
##  7  2023   356   19.4
##  8  2024   534   29.0
##  9  2025   234   12.7
## 10  2026   108    5.87
```

Note: after excluding non-personal samples, we no longer have values that are in milligrams.

We are left with the resulting microgram observations, which do appear as though they are all lacking data on air volume.

Let's do one last check, as it is possible that these values correspond to the time-weighted average:

```
dat |>
  filter(uom == "mcg") |>
  select(eighthr) |>
  group_by(eighthr) |>
  summarize(n())
```

```
## # A tibble: 2 x 2
##   eighthr `n()`
##   <chr>   <int>
## 1 N          62
## 2 Y         237
```

```
dat |>
  filter(uom =="mcg" & eighthr == "N") |>
  group_by(qualifier) |>
  summarize(n())
```

```
## # A tibble: 2 x 2
##   qualifier `n()`
##   <chr>     <int>
## 1 ND            1
## 2 ND-BLK       61
```

```
dat |>
  filter(uom =="mcg" & eighthr == "N") |>
  group_by(air_volume) |>
  summarise(
    total_rows = n(),
    non_na_values = sum(!is.na(air_volume)),
    na_count = sum(is.na(air_volume))
  )
```

```
## # A tibble: 1 x 4
##   air_volume total_rows non_na_values na_count
##        <dbl>      <int>         <int>    <int>
## 1         NA         62             0       62
```

```
dat |>
  filter(uom =="mcg" & eighthr == "N") |>
  select(result_num) |>
  summary()
```

```
##    result_num
##  Min.   :0
##  1st Qu.:0
##  Median :0
##  Mean   :0
##  3rd Qu.:0
##  Max.   :0
```

```
dat |>
  filter(uom =="mcg" & eighthr == "Y") |>
  select(result_num) |>
  summary()
```

```
##    result_num
##  Min.   : 0.00000
##  1st Qu.: 0.00000
##  Median : 0.00000
##  Mean   : 0.04304
##  3rd Qu.: 0.00000
##  Max.   :10.20000
```

Ok, we have figured out what happened –> specifically the measurements that aren't standardized to volume all contains results less than the quantification limit or are blank.

Time-weighted average observations

Next, we want to get an idea of how our samples may differ depending on how many of them used a time-weighted average.

```
dat |>
  group_by(eighthr) |>
  summarize(n())
```

```
## # A tibble: 3 x 2
##   eighthr           `n()`
##   <chr>             <int>
## 1 N                   728
## 2 STEL CEILING PEAK     8
## 3 Y                  1103
```

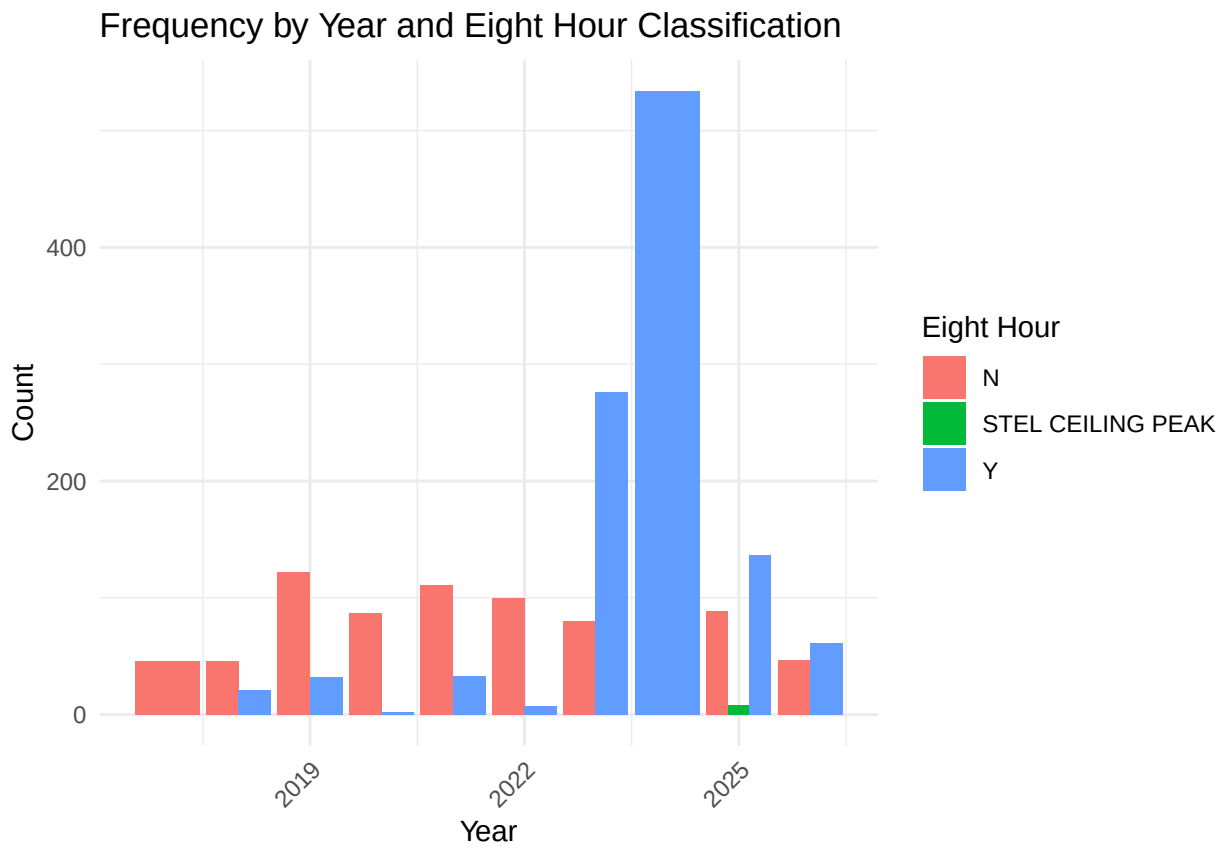
```
dat |>
  filter(eighthr == "STEL CEILING PEAK") |>
  select(everything())  # STEL information
```

```
## # A tibble: 8 x 18
##   inspection sampling_number establishment date_sampled eighthr     field_number
##        <dbl> <chr>           <chr>         <date>       <chr>       <chr>
## 1    1863156 484899-OIS      GRANITE USA   2025-12-17   STEL CEILI~ 3B1
## 2    1863156 484896-OIS      GRANITE USA   2025-12-17   STEL CEILI~ 2B2
## 3    1863156 484899-OIS      GRANITE USA   2025-12-17   STEL CEILI~ 3B2
## 4    1863156 484900-OIS      GRANITE USA   2025-12-17   STEL CEILI~ 5B1
## 5    1863156 484900-OIS      GRANITE USA   2025-12-17   STEL CEILI~ 5B2
## 6    1863156 484896-OIS      GRANITE USA   2025-12-17   STEL CEILI~ 2B1
## 7    1863156 484901-OIS      GRANITE USA   2025-12-17   STEL CEILI~ 4B1
## 8    1863156 484901-OIS      GRANITE USA   2025-12-17   STEL CEILI~ 4B2
## # i 12 more variables: type <fct>, time <dbl>, air_volume <dbl>, weight <dbl>,
## #   imis <dbl>, substance <chr>, result <chr>, uom <fct>, qualifier <chr>,
## #   year <dbl>, less_than_measurement <lgl>, result_num <dbl>
```

```r
#examining time distribution by the time-weighted data:
dat |>
  group_by(eighthr) |>
  summarise(
    count = n(),
    mean = mean(result_num, na.rm = TRUE),
    median = median(result_num, na.rm = TRUE),
    min = min(result_num, na.rm = TRUE),
    max = max(result_num, na.rm = TRUE),
    sd = sd(result_num, na.rm = TRUE)
  )
```

```
## # A tibble: 3 x 7
##   eighthr           count   mean median    min      max       sd
##   <chr>             <int>  <dbl>  <dbl>  <dbl>    <dbl>    <dbl>
## 1 N                   728 61.7   0.0126 0      4110     301.
## 2 STEL CEILING PEAK     8  0.102 0.0988 0.0302    0.195   0.0517
## 3 Y                  1103 87.2   0.0174 0      5093     382.
```

```r
#is there a time-trend component to the data that are missing time-weighting?
dat |>
  group_by(year, eighthr) |>
  summarize(count = n(), .groups = 'drop') |>
  ggplot(aes(x = year, y = count, fill = eighthr)) +
  geom_col(position = "dodge") +
  labs(title = "Frequency by Year and Eight Hour Classification",
       x = "Year",
       y = "Count",
       fill = "Eight Hour") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



Looks like we will not be retaining any of the results corresponding to the STEL ceiling peak as they have way too short times of duration.

Over 6-hours of duration

Let's create an indicator variable for whether the measurements were above 6-hours or not, and take a peek at what data remains:

```
dat <- dat |>
  mutate(six_hour = time >= 6 )

dat |>
  group_by(six_hour) |>
  summarise(
    count = n(),
    mean = mean(result_num, na.rm = TRUE),
    median = median(result_num, na.rm = TRUE),
    min = min(result_num, na.rm = TRUE),
    max = max(result_num, na.rm = TRUE),
    sd = sd(result_num, na.rm = TRUE))
```

```
## # A tibble: 3 x 7
##   six_hour count     mean  median   min    max      sd
##   <lgl>    <int>    <dbl>   <dbl> <dbl>  <dbl>   <dbl>
## 1 FALSE      817  76.7     0.0324     0 4110   259.
```

```
## 2 TRUE       631 126.     16.1        0 5093   523.
## 3 NA         391   0.0271  0          0   10.2   0.526
```

```
#what about the observations that are below, are they time-weighted?
dat |>
  filter(six_hour %in% c(FALSE, NA)) |>
  group_by(six_hour,eighthr) |>
  summarise(n())
```

```
## # A tibble: 5 x 3
## # Groups:   six_hour [2]
##   six_hour eighthr           `n()`
##   <lgl>    <chr>             <int>
## 1 FALSE    N                   323
## 2 FALSE    STEL CEILING PEAK     8
## 3 FALSE    Y                   486
## 4 NA       N                   134
## 5 NA       Y                   257
```

```
#if we only retain measurements over six hours, how much data do we have per year?
dat |>
  filter(six_hour == TRUE) |>
  group_by(year) |>
  summarise(
    count = n())
```

```
## # A tibble: 10 x 2
##     year count
##    <dbl> <int>
##  1  2017    11
##  2  2018    31
##  3  2019    40
##  4  2020    34
##  5  2021    62
##  6  2022    48
##  7  2023    97
##  8  2024   164
##  9  2025    93
## 10  2026    51
```

Before we exclude any data, we want to check if we have repeated measurements per establishments:

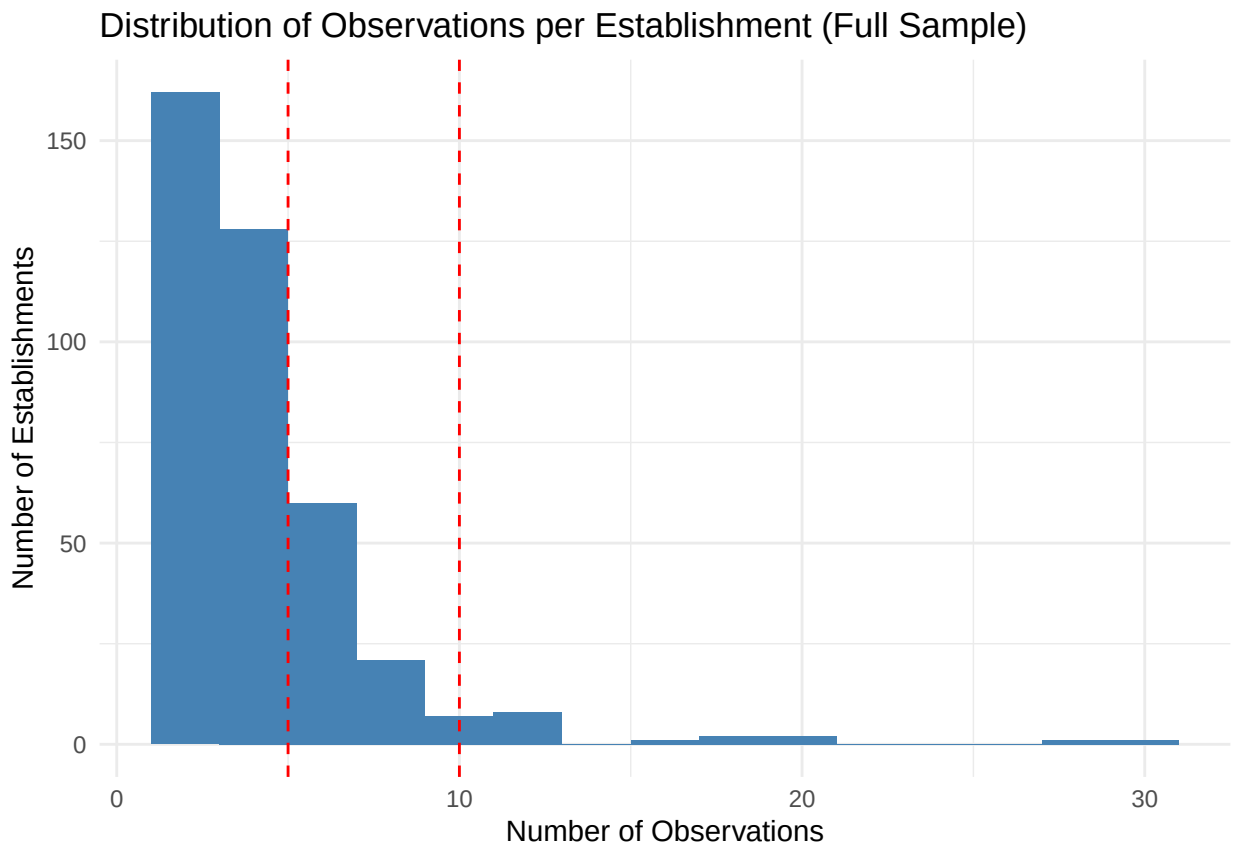
```
 full_sample_check <- dat |>
  group_by(establishment) |>
  summarise(
    n_obs = n(),
    n_years = n_distinct(year),
    .groups = 'drop'
  )

# Summary stats
full_sample_check |>
```

```
  summarise(
    total_establishments = n(),
    mean_obs_per_est = mean(n_obs),
    median_obs_per_est = median(n_obs),
    est_with_5plus = sum(n_obs >= 5),
    est_with_10plus = sum(n_obs >= 10),
    pct_5plus = round(100 * sum(n_obs >= 5) / n(), 1),
    pct_10plus = round(100 * sum(n_obs >= 10) / n(), 1)
  )
```

```
## # A tibble: 1 x 7
##   total_establishments mean_obs_per_est median_obs_per_est est_with_5plus
##                  <int>            <dbl>               <int>          <int>
## 1                  393             4.68                   4            156
## # i 3 more variables: est_with_10plus <int>, pct_5plus <dbl>, pct_10plus <dbl>
```

```
# Distribution
full_sample_check |>
  ggplot(aes(x = n_obs)) +
  geom_histogram(binwidth = 2, fill = "steelblue") +
  geom_vline(xintercept = c(5, 10), linetype = "dashed", color = "red") +
  labs(title = "Distribution of Observations per Establishment (Full Sample)",
       x = "Number of Observations",
       y = "Number of Establishments") +
  theme_minimal()
```



```r
#Restricted sample:
restricted_sample_check <- dat |>
  filter(six_hour == 1) |>  # or your > 6 hour filter
  group_by(establishment) |>
  summarise(
    n_obs = n(),
    n_years = n_distinct(year),
    .groups = 'drop'
  )

restricted_sample_check |>
  summarise(
    total_establishments = n(),
    mean_obs_per_est = mean(n_obs),
    median_obs_per_est = median(n_obs),
    est_with_5plus = sum(n_obs >= 5),
    est_with_10plus = sum(n_obs >= 10),
    pct_5plus = round(100 * sum(n_obs >= 5) / n(), 1),
    pct_10plus = round(100 * sum(n_obs >= 10) / n(), 1)
  )
```

```
## # A tibble: 1 x 7
##   total_establishments mean_obs_per_est median_obs_per_est est_with_5plus
##                  <int>            <dbl>              <int>          <int>
## 1                  207             3.05                  3             32
## # i 3 more variables: est_with_10plus <int>, pct_5plus <dbl>, pct_10plus <dbl>
```

```r
#summary table
tribble(
  ~criterion, ~guideline, ~full_sample, ~restricted_sample,
  "Number of establishments", "20-30", nrow(full_sample_check), nrow(restricted_sample_check),
  "% with 5+ observations", "~80%+",
  round(100 * sum(full_sample_check$n_obs >= 5) / nrow(full_sample_check), 1),
  round(100 * sum(restricted_sample_check$n_obs >= 5) / nrow(restricted_sample_check), 1),
  "% with 10+ observations", "~50%+",
  round(100 * sum(full_sample_check$n_obs >= 10) / nrow(full_sample_check), 1),
  round(100 * sum(restricted_sample_check$n_obs >= 10) / nrow(restricted_sample_check), 1)
)
```

```
## # A tibble: 3 x 4
##   criterion                guideline full_sample restricted_sample
##   <chr>                    <chr>           <dbl>             <dbl>
## 1 Number of establishments 20-30             393               207
## 2 % with 5+ observations   ~80%+              39.7              15.5
## 3 % with 10+ observations  ~50%+               5.6               1
```

We do not meet the assumptions of LMEM according to this article, which details that we need 5-10 observations per random-effect level for reliable variance estimates, and at least 20-30 random-effect levels.

It is recommended that no matter what, we apply cluster-robust standard errors to account for within-group variation. To deal with this, we are going to plan on fitting a generalized linear mixed model that adds a cluster-specific random effect to the linear predictor. Our cluster variable will be id.